

Fig. 5: Learning stage 1 block diagram

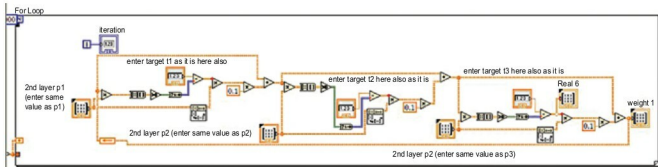


Fig. 6: Learning stage 2 block diagram

its graphical code. The block diagram contains terminal icons, nodes, wires and structures. Terminal icons are interfaces through which data is exchanged between a front panel and a block diagram. A virtual instrument is designed by going back and forth between the front panel and the block diagram, placing inputs/outputs on the front panel and building blocks on the block diagram.

In this artificial neural network, the block diagram includes real transpose matrix, nodes, wires, structures, etc.

Real transpose matrix is used to transpose the matrix having real number.

The input matrix is fed to the block and the output matrix transpose is obtained in the orange line. Errors, if any, can be observed in blue line (refer Fig. 4).

The matrix transpose output is fed to another block, which converts the real matrix from 3D into 2D array for further processing.

The feeding of inputs and initialization of weights are done in the first stage of the block (Fig. 5). That is, the first iteration is done in this stage. The entire calculation and weight inputs from the first stage are now fed to the second stage (Fig. 6) for further

processing to train the system. The iterations are completed and the final weights obtained in the second stage.

Learning algorithm

Here we explain learning algorithm for a single-layer perceptron. For multilayer perceptrons, more complicated algorithms such as backpropagation must be used. Alternatively, methods such as the delta rule can be used if the function is non-linear and differentiable, although the one below will work as well.

The learning algorithm we demonstrate is the same across all the output neurons. Everything that follows is applied to a single neuron.

We first define some variables:

$y = f(Z)$ denotes the output from the perceptron for input vector 'Z'

'b' = Bias, which in the example below is taken as 0

$D = \{(x_i, d_i), \dots, (x_n, d_n)\}$ is the training set of 's' samples, where ' x_i ' is the n-dimensional input vector and ' d_i ' is the desired output value of the perceptron for that input.

We show the values of the nodes as follows:

$x_{i,j}$ = Value of the i th node of the j th training input vector
 $x_{j,0} = 1$

Weights are represented by w_i , which is the i th value in the weight vector, to be multiplied by the value of the i th input node.

An extra dimension, with index $n+1$, can be added to all input vectors, with $x_{i,n+1} = 1$, in which case w_{n+1} replaces the bias term. To show the time-dependence of ' w ', we use $w_i(t)$, which is weight i at time t .

α = Learning rate, where $0 < \alpha \leq 1$

Too high a learning rate makes the perceptron periodically oscillate around the solution unless additional steps are taken.

Appropriate weights are applied to the inputs, and the resulting weighted sum is passed to a function that produces the output ' y '.

Learning algorithm steps. With the variables defined above, we will calculate the output as follows:

First, calculate the actual output using the function:

$$y_i(t) = f[w(t) \cdot x_i] = f[w_0(t) + w_1(t)x_{i,1} + w_2(t)x_{i,2} + \dots + w_n(t)x_{i,n}]$$

Second, adapt weights as follows: $w_i(t+1) = w_i(t) + \alpha(d_i - y_i(t))x_{i,i}$ for all nodes $0 \leq i \leq n$.

Now proceed as follows:

1. Initialise weights w_1, w_2, w_3 , and threshold, which has been done